

Creating Migrations

In the last lesson, we learned what migration are and we looked at the default migrations that come with Laravel. In this lesson, we will create our first migration.

As I mentioned in the previous lesson, there is already a `jobs` table so we can not use that for our job listings. We will instead use a table called `job_listings`.

Creating a Migration

To create a migration, we will use the `make:migration` command. This command will create a migration file in the `database/migrations` folder.

```
php artisan make:migration create_job_listings_table
```

This will create a migration file in the `database/migrations` folder with the timestamp and the name of the migration. The file will look something like this:

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('job_listings', function (Blueprint $table) {
            $table->id();
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('job_listings');
    }
};
```

This migration will create a table called `job_listings` with an `id` and `timestamps` columns. The `id` column will be the primary key and the `timestamps` columns will be used to store the created and updated timestamps.

Obviously we will want to add more fields to this table. Here is an example of a listing and the fields that we want them to have:

```
[
  [
    "id" => 1,
    "user_id" => 1,
    "title" => "Software Engineer",
    "description" => "As a Software Engineer at Algorix, you will be responsible for designing, developing, and maintaining high-quality software applications. You will work closely with cross-functional teams to deliver scalable and efficient solutions that meet business needs. The role involves writing clean, maintainable code, participating in code reviews, and staying current with industry trends to ensure our technology stack remains cutting-edge.",
    "salary" => 90000,
    "tags" => ["development", "coding", "java", "python"],
    "job_type" => "Full-time",
    "remote" => false,
    "requirements" => "Bachelors degree in Computer Science or related field, 3+ years of software development experience",
    "benefits" => "Healthcare, 401(k) matching, flexible work hours",
    "address" => "123 Main St",
    "city" => "Albany",
    "state" => "NY",
    "zipcode" => "12201",
    "contact_email" => "info@algorix.com",
    "contact_phone" => "348-334-3949",
    "company_name" => "Algorix",
    "company_description" => "Algorix is a leading tech firm specializing in innovative software solutions and cutting-edge technology.",
    "company_logo" => "logos/logo-algorix.png",
    "company_website" => "https://algorix.com"
  ]
]
```

However, at the moment, I don't want to have a ton of fields to work with as you're learning, so let's just have a few fields.

Let's add the fields to the migration file in the `up` method:

```
public function up(): void
{
    Schema::create('job_listings', function (Blueprint $table) {
        $table->id();
        $table->string('title');
        $table->text('description');
```

```
        $table->timestamps();  
    });  
}
```

For now, we will just have an id, title, description and timestamps. I don't really care about the fields at the moment. We can always add more fields later.

Running Migrations

To run the migration, we will use the `migrate` command:

```
php artisan migrate
```

This will run all of the migrations that have not been run yet, which in this case is the `create_job_listings_table` migration.

Once you migrate, you can check your database with PG Admin or another method. You should see a `job_listings` table with all of the fields that we added.

Don't do this now, but if you realize you made a mistake and let's say you forgot a field, you can rollback the migration with the following command:

```
php artisan migrate:rollback
```

This will rollback the last migration that was run. If you want to rollback all of the migrations, you can use the `reset` command:

```
php artisan migrate:reset
```